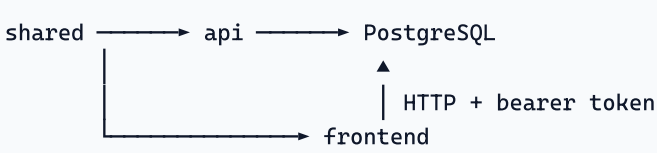
Architecture

How the three packages fit together, what crosses between them, and why the boundaries sit where they do.

The three packages



Package	Name	Responsibility
`api`	`@peoplepay360/api`	Every database read and write. Authorisation. Payroll computation.
`frontend`	`@peoplepay360/web`	Rendering, forms, navigation. Holds a session cookie and an HTTP client.
`shared`	`@peoplepay360/shared`	Types, enums, the RBAC matrix, pure domain maths.

They are npm workspaces, so `@peoplepay360/shared` resolves by symlink to the sources in this repository rather than to a published copy. Editing a shared type is immediately visible to both consumers; the API and the web client cannot drift apart on a DTO shape without a type error.

Why the API owns the database

The web client never opens a database connection. Every read goes through an HTTP endpoint that has already applied the permission matrix and, for the Employee role, narrowed the query to that person's own rows. Putting a second database client in the web app would mean a second place to get authorisation right.

What belongs in `shared`

Only things both sides genuinely need:

- **Transport types** (`types.ts`) — the DTO shapes crossing the wire. Dates are ISO strings; neither side pretends they are `Date` objects.
- **Enums** (`enums.ts`) — plain const objects rather than TypeScript `enum` s, so the values survive JSON transport unchanged and can be iterated to build a select.
- **The RBAC matrix** (`rbac.ts`) — one table, consulted by the API to enforce and by the client to decide what to offer.
- **Domain maths** (`domain.ts`) — pure calculations such as deriving weekly hours from a schedule pattern, so the client can preview a value using the exact function the server will use to persist it.

Nothing with a runtime dependency on Nest, Prisma, React or the DOM goes in here.

Request flow

A signed-in page load, end to end:

```
browser ──► Next.js server ──► NestJS ──► Prisma ──► PostgreSQL
              │  reads pp360_token cookie
              │  sends Authorization: Bearer <jwt>
              ▼
          renders on the server, streams HTML
```

1. The browser requests a route. It carries the `pp360_token` cookie, which is **httpOnly** — no client-side script can read it.
2. The Next.js server component calls `apiFetch`, which reads that cookie and forwards it as a bearer token. The token therefore never reaches the browser's JavaScript.
3. NestJS validates the token, loads the user, checks the permission matrix for the route, and narrows the query for the Employee role.
4. The page renders on the server with real data. There is no client-side data fetching layer and no loading spinner over an empty shell.

A write follows the same path through a **server action** rather than a route handler.

Sessions

Two cookies, both httpOnly, both set by the Next.js server after a successful login:

Cookie	Holds	Why
`pp360_token`	The JWT	Sent to the API as a bearer token. Never readable by scripts.
`pp360_user`	Name, email, role, employee id, `mustChangePassword`	Renders the shell without a round trip on every navigation.

`getSession()` reads the cached identity for rendering. `verifySession()` asks the API instead, and is used where a stale role would matter — the profile page, for one, so a role an admin changed shows up immediately rather than at the next sign-in. `refreshSession()` re-reads `/auth/me` and rewrites the cookie; it is what a password change calls, since the cached copy would otherwise still say the password must change and bounce them straight back.

Every guard — `requireAccess`, `requireMe`, `requireSession` — redirects to `/change-password` while `mustChangePassword` is set. A one-time password gets someone as far as choosing their own and no further, and the check lives in the guards rather than in middleware so it cannot be missed by a route that forgot to opt in.

Where a signed-in person lands is `landingFor(user)`: employees go to `/me`, everyone else to the admin panel. It is one function so login, the password change and the root route cannot disagree about it.

The API re-checks the token against the database on every request, so deactivating an account takes effect at once rather than at token expiry.

Authorisation

One matrix in `shared/src/rbac.ts` maps role → module → action:

```
can(role, 'payruns', 'create')     // boolean
visibleModules(role)               // Module[] — what appears in the nav
scopeToOwnRecords(role)            // true only for EMPLOYEE
```

Both sides read it, for different purposes:

- **The API enforces it.** `@RequirePermission('payruns', 'create')` on the controller, checked by a global guard. A route is authenticated by default and must opt out with `@Public()`.
- **The client obeys it.** `requireAccess(module)` redirects someone who cannot read a screen to the first screen they can. `can( ... )` decides whether a button is rendered at all.

The client-side check is a courtesy, not the boundary. It exists so a role is never shown a control that would fail when clicked — but removing it would change nothing about what the API permits.

Own-records scoping

The Employee role sees only rows tied to its own employee record. That narrowing happens in the API's query layer, not in the matrix, because it is a filter rather than a permission. An account with no linked employee record resolves to an id nothing matches, so it sees an empty list rather than everyone's.

Where the layers sit

Inside the API. Controllers handle HTTP and permissions. Services hold the business logic and are the only callers of Prisma. DTOs validate input at the boundary with `class-validator`, and a body carrying unknown fields is rejected outright rather than stripped.

Inside the web client.

Directory	Holds
`components/ui/`	The component library. Buttons, fields, overlays, layout. Knows nothing about the domain.
`components/data/`	Reading. Tables, filter bars, status badges, empty states, stat tiles.
`components/form/`	Writing. The record dialog, the form renderer, row menus, action buttons.
`components/app/`	The shell. Sidebar, breadcrumbs.
`lib/`	The API client, session, access rules, field specs, mutations, formatting.
`app/(app)/<domain>/`	One folder per screen: `page.tsx`, `fields.ts`, `actions.ts`, `_components/`.

A screen composes; it does not define new primitives. See frontend.md.

Pagination

Every list endpoint takes `page` and `pageSize` and returns the same envelope:

```
type Paginated<T> = { items: T[]; total: number; page: number; pageSize: number }
```

`pageArgs()` clamps the request — 20 by default, 500 at most — and `paginated()` builds the reply, so a caller cannot ask for the whole table and no endpoint invents its own defaults. The rows and the count come from one `$transaction`, so a write between them cannot produce a total that disagrees with the page.

Filtering and scoping happen in the same query. That matters more than the page size: the Employee role's own-records restriction is applied at the source, so a page of results never holds rows the browser then has to be trusted to hide.

Ids

Every primary key is a **UUIDv7** (`@default(uuid(7))`) — globally unique but time-sortable, unlike UUIDv4, so index locality stays good.

Ids are still **not** validated with `IsUUID` / `ParseUUIDPipe` . This bit once, when the keys were cuids and every by-id route returned 400; the validator that replaced it is deliberately kept even though the keys are now genuinely UUIDs, for two reasons. The id-bearing columns that carry no foreign key — `AuditLog.entityId` , `Attendance.editedById` , the `approvedBy` / `refusedBy` / `paidBy` fields — are plain text and are not guaranteed to be UUIDs at all. And a validator that does not hard-code the id format means changing strategy again would not mean editing every DTO.

So `api/src/common/validation/entity-id.ts` provides `IsEntityId` and `ParseEntityIdPipe` , matching a loose `^[A-Za-z0-9_-]{16,64}$` — enough to reject empty strings and obvious junk. There are no `IsUUID` or `ParseUUIDPipe` left in the modules, and none should return.

Money and time

- **Money and hours** are PostgreSQL `NUMERIC` , never floats, and are normalised to plain numbers at the API boundary. Payroll totals reconcile to the cent.
- **Dates** cross the wire as ISO strings. Date-only columns are rendered in UTC so the server and client cannot disagree and hydration cannot mismatch.
- **Instants** — attendance punches — are shown in UTC everywhere, including in the form that edits them, and the field says so. The alternative, showing local time in one place and UTC in another, made the same punch read two different ways on one screen.